

Secure Channel with libsodium

The complete project guide — every part explained, from the cryptography basics to the last file

Project	Secure Channel // Cryptographic Command Center
Author	Pratham (individual project)
Submission	24 October 2026
Languages	C11 (cryptography), Node.js (local bridge), HTML/CSS/JS (dashboard), Bash
Main library	libsodium 1.0.18 · compared with OpenSSL 3.5.5
Location	D:\project related 24 oct\secure-workshop

How to read this guide: Chapters 1–3 explain what the project is and the cryptography you need to understand it. Chapters 4–6 explain the structure and how it is built. Chapters 7–12 walk through the six required tasks. Chapters 13–19 cover the extensions, the engine, the bridge, the dashboard and the tests. Chapters 20–23 tell you how to run it, answer the examiner's likely questions, and list the limitations.

Contents

1	The project in one page	13	The session state machine
2	Cryptography you need to know	14	Extension: signed handshake vs MITM
3	Threat model: what we defend against	15	Extensions: re-keying, TCP, benchmark
4	Architecture: how the pieces fit	16	sc_engine: the machine interface
5	Every file, explained	17	The Node bridge and its security
6	Building: the Makefile	18	The dashboard and Demo Mode
7	Task 1 — Key generation and storage	19	Testing: four suites
8	Task 2 — Key exchange	20	How to run everything
9	Task 3 — Packets and AEAD	21	Design questions and answers
10	Task 4 — Replay protection and the 8 attacks	22	Limitations
11	Task 5 — Key wrapping with Argon2id	23	Quick-reference cheat sheet
12	Task 6 — OpenSSL comparison		

1 The project in one page

This project builds a **secure communication channel**: a way for a client and a server to send messages to each other so that someone listening on the network cannot read them, cannot change them, and cannot replay old ones. It is written in C using **libsodium**, a modern cryptography library designed to be hard to misuse. The project then compares that design with the **OpenSSL command-line tool** and shows, with a live attack, why the OpenSSL workflow in the brief is weaker.

The three guarantees a secure channel must give

Guarantee	Meaning in plain words	How this project achieves it
Confidentiality	Nobody on the network can read the message.	ChaCha20 encryption with a 256-bit session key.
Integrity / authenticity	Nobody can change a message (or invent one) without the receiver noticing.	A 16-byte Poly1305 authentication tag on every packet.
Freshness	An old message cannot be sent again (replay), and messages cannot be swapped in order.	A 64-bit sequence number, authenticated, that must always increase.

What was built

- **Six required tasks** from the brief: key generation (1), key exchange (2), authenticated encryption (3), replay protection with attack tests (4), password-based key wrapping (5), and the OpenSSL comparison (6).
- **Four optional extensions**: a signed handshake that stops a man-in-the-middle, automatic re-keying, a real TCP client/server on localhost, and a speed benchmark.
- **A web dashboard** (the "Cryptographic Command Center") that runs the *real C* programs and shows their results. Nothing on screen is faked: every PASS, BLOCKED or ACCEPTED is a return code from C.
- **A terminal demo** (`secure_demo`) so the project can be presented without a browser.
- **Four test suites**: the brief's 8 attack tests, 35 extended C tests, 46 integration tests for the dashboard and bridge, and the OpenSSL comparison script.

The one-sentence summary to remember

ELEVATOR PITCH

Two parties agree on fresh keys with X25519, every message is encrypted and authenticated with ChaCha20-Poly1305 with the sequence number bound as extra authenticated data, the receiver changes its replay state only after the tag verifies, long-term private keys are stored 0600 and can be wrapped under a passphrase with Argon2id — and a dashboard proves all of it live against the real code.

Recorded results

These figures come from the project's own runs (README, `docs/report.md` , `docs/output.txt`) on Ubuntu under WSL2 with gcc 15.2.0, libsodium 1.0.18, OpenSSL 3.5.5 and Node 24. Run the commands yourself before presenting; the dashboard shows *NOT RUN* until you do.

Check	Command	Recorded result
Build	<code>make</code>	11 programs, 0 warnings
Required tests (brief)	<code>make test</code>	8 passed, 0 failed + Task 1/2/5 checkpoints
Extended C tests	<code>./test_extended</code>	35 passed, 0 failed

OpenSSL comparison	<code>make openssl</code>	CBC tamper undetected; AEAD rejects
Dashboard + bridge	<code>npm run test:ui</code>	46 passed, 0 failed

2 Cryptography you need to know

Every term used later in this guide is explained here first. If you understand this chapter, you can explain the whole project.

2.1 Keys: symmetric and asymmetric

Symmetric cryptography uses one secret key shared by both sides. It is fast and is used to encrypt the actual messages (ChaCha20-Poly1305 here). The problem is: how do two parties get the same secret key without sending it?

Asymmetric (public-key) cryptography gives each party a **key pair**: a *public key* that anyone may see and a *private (secret) key* that must never leave its owner. It solves the key-sharing problem and allows signatures.

2.2 X25519 — key exchange

X25519 is an elliptic-curve Diffie–Hellman function (RFC 7748). Each side has a key pair. If the client combines *its private key* with *the server's public key*, and the server combines *its private key* with *the client's public key*, both get the **same shared secret** — yet the secret never crossed the network. An eavesdropper who sees only the two public keys cannot compute it. In libsodium this is wrapped by `crypto_kx`.

2.3 Ed25519 — digital signatures

Ed25519 (RFC 8032) lets the owner of a private key produce a 64-byte **signature** over some data. Anyone with the public key can verify it, but only the private-key holder could have made it. It proves *who* sent something. X25519 and Ed25519 use the same underlying curve (Curve25519) but have different jobs: X25519 = agree on a secret, Ed25519 = prove identity.

2.4 Hash functions and BLAKE2b

A hash turns any input into a fixed-size "fingerprint". It is one-way (you cannot recover the input) and tiny input changes give a completely different output. BLAKE2b is the hash libsodium uses. In this project it is used inside `crypto_kx` and to make 8-byte **key fingerprints** for the dashboard (so we can show two keys are equal without showing the keys).

2.5 AEAD — Authenticated Encryption with Associated Data

AEAD is one operation that does two things at once: it **encrypts** the message and produces an **authentication tag**. The tag covers the ciphertext *and* some extra data that is sent in clear (the "associated data", AAD). On decryption, the receiver recomputes the tag; if even one bit of the ciphertext or AAD has changed, the tag does not match and decryption **refuses to return anything**.

- **ChaCha20** is the cipher: a stream cipher that generates a keystream which is XORed with the plaintext. Fast and constant-time on any CPU.
- **Poly1305** is the authenticator that produces the 16-byte tag.
- Together, **ChaCha20-Poly1305-IETF** (RFC 8439) uses a 32-byte key and a 12-byte nonce.

2.6 Nonce

A **nonce** ("number used once") is a value that makes each encryption unique, so encrypting the same message twice gives different ciphertexts. It is not secret. It must **never repeat under the same key**: with a stream cipher,

reusing a nonce reuses the keystream, so XORing two ciphertexts reveals the XOR of the two plaintexts, and Poly1305 tags become forgeable. This project uses 12 random bytes per packet.

2.7 AAD — Associated Data

Data that must travel in clear but must not be changed. Here, the **sequence number** is the AAD: the receiver must read it before decrypting (to reject replays cheaply), but because it is covered by the tag, an attacker cannot alter it.

2.8 KDF, Argon2id and salt

A **Key Derivation Function** turns some input into a key. Passwords are weak and guessable, so a password KDF must be deliberately **slow** and **memory-hungry** to make each guess expensive. **Argon2id** (RFC 9106; `crypto_pwhash`) is the modern choice. Here it uses 64 MiB of RAM per guess, which stops GPUs from testing thousands of passwords in parallel cheaply. A **salt** is random data mixed in so the same password gives a different key for each file, defeating precomputed tables.

A second, fast KDF — `crypto_kdf_derive_from_key` — is used for re-keying: it derives a new key from an old key plus a counter.

2.9 secretbox

`crypto_secretbox` is libsodium's simple symmetric authenticated encryption (XSalsa20-Poly1305, 24-byte nonce, 16-byte MAC). Task 5 uses it to encrypt the private key file under the Argon2id-derived key.

2.10 Other ideas used everywhere

Term	Meaning
Forward secrecy	Session keys come from <i>ephemeral</i> (throw-away) key pairs that are wiped after use. Stealing a long-term key later does not decrypt old sessions.
Man-in-the-middle (MITM)	An attacker who sits between client and server and pretends to be each one to the other. Plain key exchange cannot detect this; signatures can.
Replay attack	Re-sending a genuine packet captured earlier (e.g. "pay Bob 100" twice).
Reorder attack	Delivering genuine packets in a different order than they were sent.
Constant-time comparison	<code>sodium_memcmp</code> takes the same time whatever the data, so timing does not leak where two keys differ. Normal <code>memcmp</code> stops at the first difference.
<code>sodium_memzero</code>	Wipes memory in a way the compiler cannot optimise away. A plain <code>memset</code> on a buffer that is never read again can be deleted as a "dead store", leaving the key in RAM.
<code>sodium_malloc</code>	Allocates memory with guard pages, locked so it is not swapped to disk; <code>sodium_free</code> zeroes it before releasing.
File mode 0600 / 0644	Unix permissions. 0600 = <code>-rw-----</code> : only the owner can read/write (private keys). 0644 = <code>-rw-r--r--</code> : everyone can read (public keys).
Big-endian	Most significant byte first. The sequence number is stored this way so both sides read it identically.
CBC mode	An older block-cipher mode (used by <code>openssl enc -aes-256-cbc</code>) that encrypts but has no tag — so it gives confidentiality only, not integrity.

3 Threat model: what we defend against

A threat model says precisely what the attacker can and cannot do. Every design choice in the project answers one line of it.

The attacker controls the network. They can:

Capability	Defence in this project
Read every packet	ChaCha20 encryption; keys never travel.
Modify any byte	Poly1305 tag over ciphertext and sequence number → rc -2.
Replay an old packet	Sequence must be strictly greater than <code>last_seq</code> → rc -3.
Reorder packets	Same replay window: an older seq after a newer one → rc -3.
Drop packets	Cannot be prevented by cryptography; the design only guarantees a dropped packet is never silently replaced by a forged one.
Inject packets of any length or content, including forged sequence numbers	Length check (rc -1), tag check before any state change (rc -2).
Reflect a packet back to its sender	Separate keys per direction (<code>client_tx</code> ≠ <code>client_rx</code>).

The attacker cannot:

- read files protected by 0600 permissions on the server's disk;
- know the Argon2id passphrase. (A stolen disk or backup is handled by Task 5 key wrapping.)

HONEST GAP

Plain `crypto_kx` , as required by the brief, does **not** stop an *active* man-in-the-middle during the handshake: nothing proves who owns the public key you received. The project demonstrates this attack and fixes it in the signed-handshake extension (Chapter 14).

4 Architecture: how the pieces fit

The project has three layers. The cryptography lives only in C. Node.js is just a messenger. The browser only displays.

Browser (Windows)

- `dashboard/` — static HTML, CSS, JS
- No CDN, no framework
- Calls the bridge with HTTP; receives live events by Server-Sent Events (SSE)
- Never computes a cryptographic result

Bridge (Node, Windows)

- `bridge/server.js` , zero dependencies
- Listens on 127.0.0.1:8787 only
- Whitelists and validates every command
- Runs C programs through `WSL -d Ubuntu --`

C programs (WSL Ubuntu)

- `sc_engine` — long-running, holds the live session
- `keygen` , `test_attacks` , `test_extended` , `aead_bench` , OpenSSL script
- `sc_server` / `sc_client` on TCP :7700

What happens when you click a button — step by step

Example: you click **Tamper** in the Attack Lab.

1. The browser sends `POST /api/engine` with body `{"cmd":"attack","arg":"tamper"}` and the header `X-SC-Client: dashboard`.
2. The bridge checks the `Host` header, the custom header and the content type. It checks `tamper` is in its list of 8 allowed attacks and turns it into the text line `attack tamper`.
3. The bridge writes that line to the standard input of the running `sc_engine` process inside WSL.
4. `sc_engine` seals a real packet with the client key, flips one bit, and calls the real `unseal()` with the server key. `unseal()` returns `-2`.
5. The engine prints one JSON line: the steps, the packet's public bytes, `"rc":-2`, `"result":"BLOCKED"`, `"pass":true`.
6. The bridge passes that JSON back to the browser, which draws it. The `BLOCKED` label is the return code.

CORE RULE: SECRETS NEVER LEAVE C

The engine keeps session keys in `sodium_malloc()` memory and emits only public keys, nonces, ciphertext, return codes, file metadata and one-way 8-byte BLAKE2b fingerprints. The integration test scans every API response for private-key bytes and passphrases.

Why Windows + WSL?

The development machine is Windows, which has no native gcc. All C code is built and run inside **WSL Ubuntu**. The Node bridge runs on Windows and starts each C program with `ws1 -d Ubuntu -- env NO_COLOR=1 ./program`. On Linux or macOS the bridge runs the binaries directly.

WSL PERMISSION DETAIL

On a Windows drive (`/mnt/d`), WSL ignores `chmod` unless the drive is mounted with the `metadata` option. Without it, `./keygen` correctly reports **FAIL** for 0600. The fix (already applied on this machine) is `[automount] options = "metadata"` in `/etc/wsl.conf`, then `ws1 --shutdown`.

5 Every file, explained

5.1 Top level

File / folder	Purpose
<code>../package.json</code> (parent folder)	Shortcuts so <code>npm run dev</code> , <code>npm test</code> etc. work from <code>D:\project related 24 oct</code> ; they forward into <code>secure-workshop/</code> .
<code>package.json</code>	npm scripts: <code>dev</code> (start bridge + open browser), <code>start</code> (no browser), <code>test</code> , <code>check</code> , <code>test:ui</code> , <code>audit</code> , <code>terminal</code> . Requires Node ≥ 18 . No dependencies.
<code>Makefile</code>	Builds all 11 C programs and defines <code>test</code> , <code>check</code> , <code>demo</code> and other targets (Chapter 6).
<code>README.md</code>	Overview, architecture, results, build and demo instructions.
<code>.gitignore</code> , <code>.gitattributes</code>	Keep binaries and generated keys out of git; force LF line endings (the Makefile needs real tabs and Unix newlines).
<code>keys/</code>	Generated key files. Only <code>.gitkeep</code> is ever committed.
<code>openssl_demo/</code>	Files produced by the OpenSSL script (PEM keys, encrypted messages). Regenerated on each run.

docs/	report.md (academic report), slides.md (slide content), presentation_qa.md (Q&A prep), output.txt (captured make test), screenshots/ (22 real captures).
-------	---

5.2 C source — src/

File	What is inside
common.h	The single source of truth for every length (key 32, seq 8, nonce 12, tag 16, header 20, minimum packet 36, max plaintext 64 KiB), key-file paths, return codes SC_OK... SC_ERR_BUFFER, the channel_state enum, and the replay_state struct.
keyio.c	Safe file I/O: write_file_0600(), write_file_pub() (0644), exact-size read_file(), file_mode(), mode_string() (0600 → "rw-----").
aead.c	seal() and unseal() — the heart of the record layer; big-endian helpers; unseal_strerror().
handshake.c/.h	Signed-handshake transcript, hs_sign() / hs_verify(), identity_load(), rekey_key(), key_fingerprint(), and the MITM scenario mitm_run().
session.c/.h	The INIT → AUTH → SECURE → TERMINATE state machine used by the engine, the terminal demo and the tests.
wrap.c/.h	Task 5: Argon2id + secretbox wrapping of the private key; 120-byte file layout.
json.c/.h	A tiny JSON writer (fixed buffer, proper escaping) so C can print one JSON object per line.
ui.c/.h	Terminal formatting: banners, boxes, [✓ PASS] lines, hex previews, state bar; honours NO_COLOR.
keygen.c	Task 1 program.
kx_demo.c	Task 2 program + Task 3 round-trip + reflection test.
keywrap.c	Task 5 program; --interactive asks for your own passphrase with echo off.
mitm_demo.c	Stretch 2: shows MITM on plain kx, then three variants blocked by the signed handshake.
benchmark.c	Stretch 4: aead_bench timing ChaCha20-Poly1305, AES-256-GCM and seal().
demo.c	secure_demo : menu, guided tour (--present , --auto) and security audit (--audit).
engine.c	sc_engine : line-in / JSON-out interface for the dashboard (Chapter 16).
net.c/.h	Localhost TCP transport with length-prefixed frames.
tcp_server.c , tcp_client.c	sc_server / sc_client : the secure channel over real sockets.

5.3 Tests, bridge, scripts, dashboard

File	Purpose
tests/test_attacks.c	The brief's 8 attack tests (Task 4). --tap gives machine-readable output.
tests/test_extended.c	35 extra C tests: packets, key files, wrapping, state machine, re-key, signed handshake.
bridge/server.js	HTTP server + API + process manager for the C binaries.
bridge/runner.js	Helper for running the C programs (WSL/native spawn).

bridge/test_integration.js	46 end-to-end checks of the bridge, API, security controls, live attacks, TCP and secret-leak scan.
scripts/openssl_compare.sh	Task 6. Normal, <code>--quiet</code> and <code>--json</code> modes.
scripts/web_demo.sh	Helper for running the terminal demo (used by <code>npm run terminal</code>).
dashboard/index.html	The single page shell with navigation for 12 views.
dashboard/css/app.css	Dark "liquid-glass" style. Contains a global <code>animation: none; transition: none</code> rule — the UI is deliberately static.
dashboard/js/app.js	Start-up, routing between views, status bar.
dashboard/js/api.js	The <code>fetch</code> wrapper that adds the required header.
dashboard/js/store.js	Shared state; every value is filled only from real responses (otherwise NOT RUN).
dashboard/js/ui.js	Helpers: <code>chip()</code> status badges, <code>emptyState()</code> , HTML escaping.
dashboard/js/trace.js	"Background activity" trace: shows the exact commands the bridge ran.
dashboard/js/demo.js	The 20-step Demo Mode.
dashboard/js/views/*.js	One file per view: overview, handshake, channel, inspector, attacks, tests, vault, wrap, openssl, bench, logs, limits.

6 Building: the Makefile

6.1 Compiler flags and why each one is there

```
CPPFLAGS = -D_POSIX_C_SOURCE=200809L -D_FORTIFY_SOURCE=2
CFLAGS   = -std=c11 -Wall -Wextra -Wpedantic -Wshadow -Wformat=2 -O2 -fstack-protector-strong
LDLIBS   = $(pkg-config --libs libsodium) # falls back to -lsodium
```

Flag	Purpose
<code>-std=c11</code>	Standard C, no compiler extensions.
<code>-Wall -Wextra -Wpedantic</code>	Turn on almost all warnings. The project builds with zero .
<code>-Wshadow</code>	Warn when a variable hides another of the same name (a common bug source).
<code>-Wformat=2</code>	Strict checks on <code>printf</code> -style format strings.
<code>-O2</code>	Optimise. (This is why <code>sodium_memzero</code> matters — the optimiser can delete a plain <code>memset</code> .)
<code>-fstack-protector-strong</code>	Stack canaries that detect buffer overflows at run time.
<code>-D_FORTIFY_SOURCE=2</code>	Extra bounds checking in libc functions.
<code>-D_POSIX_C_SOURCE=200809L</code>	Exposes POSIX functions like <code>fchmod</code> , <code>clock_gettime</code> .

6.2 The 11 programs

All programs link the same core library files: `keyio.c` `aead.c` `ui.c` `handshake.c` `wrap.c` `session.c` `json.c`. The TCP programs add `net.c`.

Required (4)	Extra (7)
--------------	-----------

keygen , kx_demo , keywrap , test_attacks	secure_demo , mitm_demo , aead_bench , sc_engine , sc_server , sc_client , test_extended
--	---

6.3 Make targets

Target	What it does
make / make all	Build all 11 programs.
make test	The brief's checkpoints, in order: ./keygen && ls -l keys/ , ./kx_demo , ./test_attacks , ./keywrap .
make check	make test + ./test_extended + the OpenSSL script in quiet mode.
make openssl	Task 6 with full readable output.
make demo / present / auto / audit	Terminal demo: menu / guided tour with Enter between steps / tour without pauses / 16-row security audit.
make mitm	MITM attack on plain kx, then the signed fix.
make benchmark	AEAD speed on this machine.
make tcp-server / tcp-client	TCP mode (server re-keys every 3 messages).
make output	Save make test output to docs/output.txt without colours.
make env	Print gcc, libsodium and OpenSSL versions.
make clean	Delete binaries, generated keys and openssl_demo/ .

7 Task 1 — Key generation and storage

Goal: create the server's long-term key pairs and store them so that private keys are readable only by their owner.
Program: keygen (src/keygen.c , using src/keyio.c).

7.1 What keygen does, in order

- sodium_init() — aborts if it fails; no key is generated without a working random-number generator.
- Creates the keys/ directory with mode 0700.
- Generates an **Ed25519** signing pair (crypto_sign_keypair) and an **X25519** pair (crypto_kx_keypair). Prints only the first bytes of the public keys.
- Writes four files (table below).
- Wipes the secret buffers with sodium_memzero() immediately.
- Checks each file's mode with stat() ; prints PASS or FAIL.
- Functional reload test:** reads the Ed25519 secret key back, signs a probe message, and verifies it with the public key. Reads the X25519 secret back and recomputes the public key with crypto_scalarmult_base ; it must match.
- Prints a summary box and Keys generated and loaded OK ; exits 0 only if everything passed.

File	Content	Size	Mode
keys/server_pk.bin	Ed25519 public key	32 B	0644

keys/server_sk.bin	Ed25519 secret key (seed public key)	64 B	0600
keys/server_kx_pk.bin	X25519 public key	32 B	0644
keys/server_kx_sk.bin	X25519 secret key	32 B	0600

7.2 The key line of code — atomic creation

```
int fd = open(path, O_WRONLY | O_CREAT | O_TRUNC | O_NOFOLLOW | O_CLOEXEC, mode);
if (fd < 0) return -1;
if (fchmod(fd, mode) != 0 || write_all(fd, buf, len) != 0) { close(fd); return -1; }
```

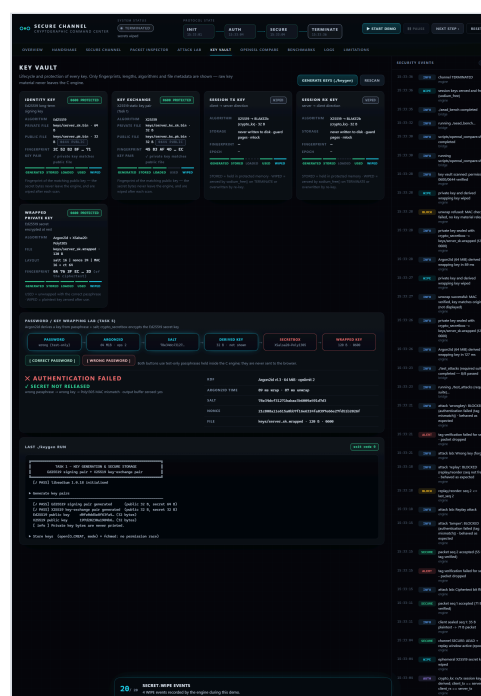
- **Mode given to `open()`** : the file is *born* 0600, so there is never a moment it is readable by others. With `fopen()` then `chmod()`, another user could open the file in the gap and keep reading through their file descriptor.
- **`fchmod()`** : still needed because `open()` ignores the mode if the file already exists, and the umask can remove bits.
- **`O_TRUNC`** : empties an old file before the secret is written.
- **`O_NOFOLLOW`** : refuses to write through a symbolic link an attacker may have planted (e.g. pointing to a file they can read).
- **`O_CLOEXEC`** : the file descriptor is not inherited by child processes.

7.3 Exact-size reading

`read_file()` refuses any file whose size is not *exactly* the key length, and refuses non-regular files. If a read is cut short, the partial buffer is wiped. A truncated or padded file can never be half-loaded as a key.

7.4 Checkpoint output

```
-rw-r--r-- 1 pratham_gosai pratham_gosai 32 Sep 30 15:38 server_kx_pk.bin
-rw----- 1 pratham_gosai pratham_gosai 32 Sep 30 15:38 server_kx_sk.bin
-rw-r--r-- 1 pratham_gosai pratham_gosai 32 Sep 30 15:38 server_pk.bin
-rw----- 1 pratham_gosai pratham_gosai 64 Sep 30 15:38 server_sk.bin
```



Key Vault view: file modes and key-pair consistency checked by the C code.

8 Task 2 — Key exchange and session keys

Goal: client and server derive shared session keys without ever sending them. **Program:** `kx_demo`.

8.1 Step by step

1. **INIT** — each side creates a fresh (ephemeral) X25519 key pair. Only the public keys "cross the network".
2. **AUTH** — the client calls `crypto_kx_client_session_keys(c_rx, c_tx, cpk, csk, spk)`; the server calls `crypto_kx_server_session_keys(s_rx, s_tx, spk, ssk, cpk)`.
3. Internally `crypto_kx` computes `BLAKE2b-512( X25519(sk, peer_pk) || client_pk || server_pk )` and splits the 64 bytes into two 32-byte keys. Client and server take the halves in **opposite order**.
4. The ephemeral secret keys are wiped as soon as the session keys exist (forward secrecy).
5. The keys are compared with constant-time `sodium_memcmp()`. Key bytes are never printed.

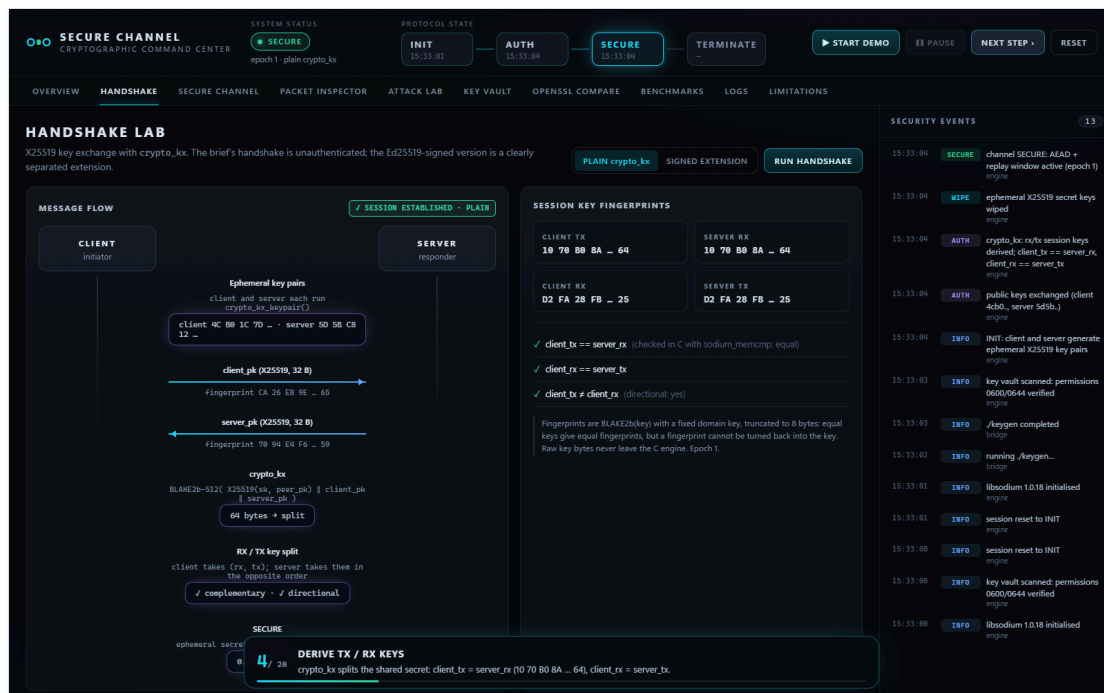
```
client_tx == server_rx : YES
client_rx == server_tx : YES
client_tx == client_rx : NO (should be NO)
```

8.2 Why two keys per direction?

What the client transmits with, the server receives with — and the reverse direction uses a different key. So if an attacker **reflects** a client packet back to the client, the client tries to open it with `c_rx`, which is not the key it was sealed with: `rc -2`. `kx_demo` tests exactly this ("Reflection attack ... rejected").

8.3 Task 3 checkpoint inside `kx_demo`

The client seals "hello secure world" (18 B) with `seq = 1` → a 54-byte packet (20 header + 18 ciphertext + 16 tag). The server opens it with `s_rx` and prints `Round-trip: "hello secure world"`. At the end every key is wiped and the state bar shows **TERMINATE**.



Handshake view: matching 8-byte fingerprints prove `client_tx = server_rx` without revealing the keys.

9 Task 3 — Packets and authenticated encryption

Goal: turn each message into an encrypted, authenticated packet. **Code:** `seal()` and `unseal()` in `src/aead.c`.

9.1 Packet format

seq	nonce	ciphertext	tag
8 B, big-endian clear, but AAD	12 B random per packet	n B (same length as plaintext) ChaCha20	16 B Poly1305

header = 20 B · minimum packet = 36 B · total = 36 + n bytes

Example: "Meet at the library at 10:00" (28 B) → 20 + 28 + 16 = **64-byte packet**.

9.2 `seal()` — sending

```
store_be64(seqb, seq); /* 1. write seq, big-endian */
randombytes_buf(nonce, NONCE_LEN); /* 2. fresh random 12-byte nonce */
crypto_aead_chacha20poly1305_ietf_encrypt(ct, &ct_len, pt, pt_len,
    seqb, SEQ_LEN, /* 3. AAD = seq: clear text, but authenticated */
    NULL, nonce, key);
```

`seal()` also takes the output buffer capacity and refuses to write if it is too small, and refuses plaintext over 64 KiB. It returns the packet length, or 0 on error.

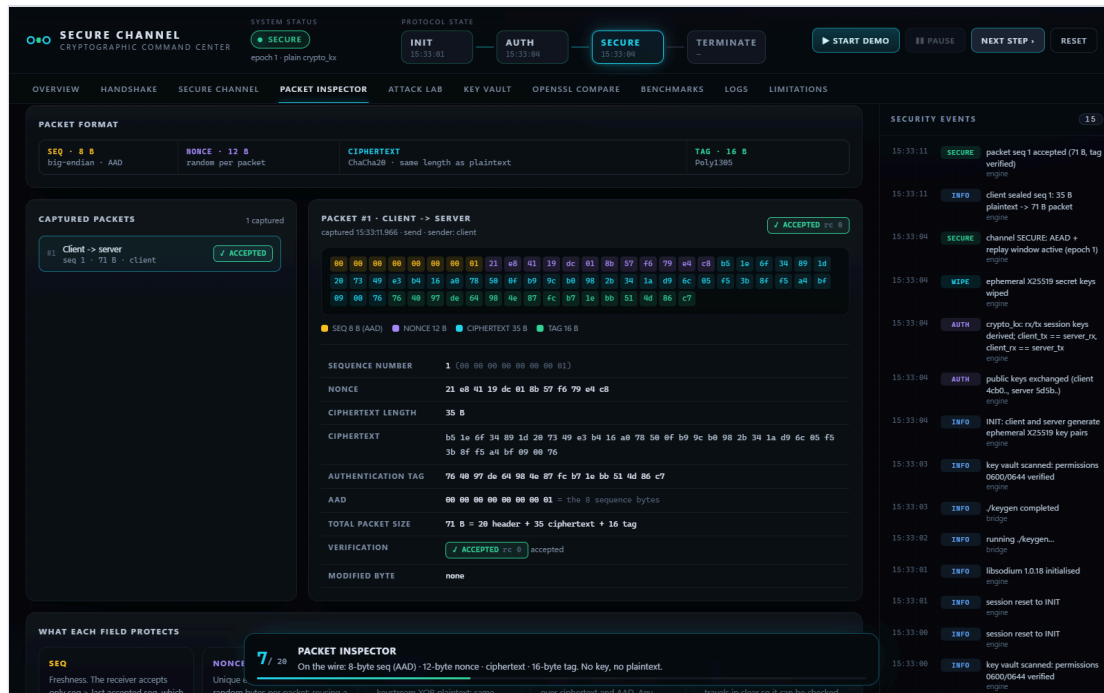
9.3 `unseal()` — receiving, in a fixed order

1. **Length check** — shorter than 36 B or longer than the maximum → **-1**, before any cryptographic work.
2. **Buffer check** — caller's plaintext buffer too small → **-4**.
3. **Replay check (read-only)** — read seq; if a packet was already accepted and `seq ≤ last_seq` → **-3**. The state is only *read* here.
4. **Tag check + decrypt** — verify Poly1305 over ciphertext and AAD (seq). Failure → wipe output buffer, return **-2**.
5. **Only now** update `last_seq = seq` and return the plaintext with **0**.

rc	Name	Meaning
0	SC_OK	Accepted
-1	SC_ERR_SHORT	Too short / too long / malformed
-2	SC_ERR_AUTH	Tag did not verify (tamper, wrong key, forged seq)
-3	SC_ERR_REPLAY	seq not strictly greater than the last accepted one
-4	SC_ERR_BUFFER	Caller's output buffer too small

9.4 Hardening changes to the guide's starter code

1. `seal()` / `unseal()` take explicit output capacities so a caller can never overflow a buffer; messages over 64 KiB are rejected; extra code **-4**.
2. `read_file()` rejects key files that are not exactly the right size.
3. Key files use `O_NOFOLLOW` and `O_CLOEXEC`; public keys get an explicit `fchmod(0644)`.
4. On authentication failure, `unseal()` zeroes the output buffer.
5. Task 1's loader test is functional (a real signature must verify).



Packet Inspector: exactly what an eavesdropper sees — seq, nonce, ciphertext, tag. No key, no plaintext.

10 Task 4 — Replay protection and the 8 attacks

10.1 The replay window

The receiver keeps `replay_state { last_seq, has_seq }`. A packet is accepted only if its seq is **strictly greater** than the last accepted one. This rejects both replays (same seq again) and reordering (an older seq after a newer one).

THE MOST IMPORTANT RULE IN THE PROJECT

`last_seq` is updated **only after the tag verifies**. Until then the seq field is just bytes the attacker chose. If the state were updated first, one forged packet with `seq = 264-1` would fail authentication but still push `last_seq` to the maximum, and every genuine packet afterwards would be rejected as a replay — a one-packet denial of service. Only authenticated data may change protocol state.

10.2 The 8 automated tests (`tests/test_attacks.c`)

Each test uses a fresh replay state and a fresh packet. `make test` prints a boxed result and must end **8 passed, 0 failed**.

#	Test	What the attacker does	Expected
1	Round-trip	Nothing — seal then unseal	plaintext matches, rc 0
2	Tamper	Flip one ciphertext byte	rc -2
3	AAD tamper	Change the seq bytes	rc -2
4	Replay	Send the same packet twice	second: rc -3
5	Reorder	Deliver seq 3, then seq 2	seq 2: rc -3
6	Wrong key	Decrypt with another key	rc -2

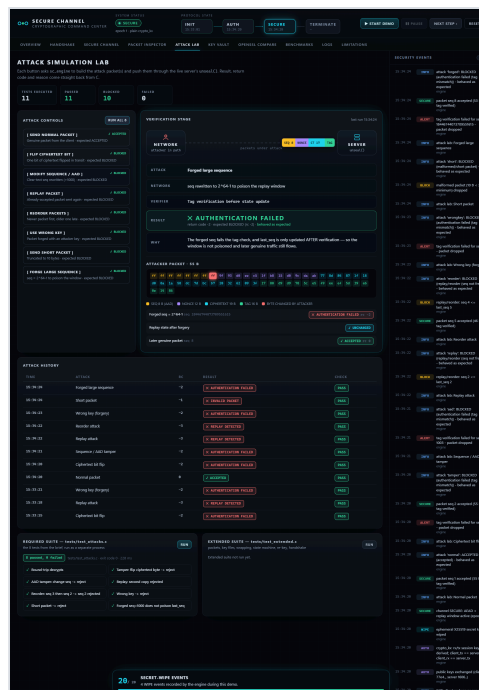
7	Short packet (extra)	Send a 10-byte packet	rc -1
8	Forged seq (extra)	Tampered packet with seq = 1000, then genuine seq 5	seq 5 still accepted; has_seq still 0 after the forgery

10.3 The same attacks live in the dashboard Attack Lab

The engine runs each attack against the *live* session (`engine.c` , `cmd_attack`). Some do more than the C test to prove side effects:

Attack	What the engine actually does	rc
normal	Seals "transfer 100 to Bob" and delivers it.	0
tamper	Flips bit 0 of the first ciphertext byte; checks the replay state did not move; then delivers the original — still accepted.	-2
aad	Rewrites the clear-text seq to seq + 1000.	-2
replay	Re-sends a copy of the last accepted packet.	-3
reorder	Seals message #1 and #2, delivers #2 first (accepted), then #1.	-3
wrongkey	Seals with a freshly generated attacker key.	-2
short	Truncates a packet to 10 bytes.	-1
forged	Sets seq = $2^{64}-1$; reports "Replay state after forgery: UNCHANGED"; then a genuine follow-up packet is accepted.	-2

An attack is marked **pass** only if the primary rc equals the expected rc *and* every side-check holds. Outside the SECURE state the engine refuses to run an attack.



Attack Lab after RUN ALL 8: seven attacks BLOCKED, the normal packet ACCEPTED — all return codes from `unseal()` .

11 Task 5 — Key wrapping with Argon2id

Problem: a private key protected only by file permissions is exposed if the disk or a backup is stolen. **Solution:** encrypt it under a passphrase. **Code:** `src/wrap.c` , program `keywrap` .

11.1 Wrapping, step by step

- 1. Generate a random 16-byte **salt** and a random 24-byte **nonce**.
- 2. **Argon2id** (`crypto_pwhash` , `OPSLIMIT_INTERACTIVE` , `MEMLIMIT_INTERACTIVE` = 64 MiB) turns passphrase + salt into a 32-byte key.
- 3. `crypto_secretbox_easy` (XSalsa20-Poly1305) encrypts the 64-byte Ed25519 secret key.
- 4. The derived key is wiped; the result is written to `keys/server_sk.wrapped` with mode 0600.

salt 16 B	nonce 24 B	MAC 16 B	encrypted secret key 64 B
total = 120 bytes			

11.2 Unwrapping

Read the salt and nonce, re-derive the key with Argon2id, and open the box. A **wrong passphrase** gives a different key, so the MAC check fails, the output buffer is wiped, and **no key material is returned**. A tampered file fails the same way.

11.3 Self-test results printed by `keywrap`

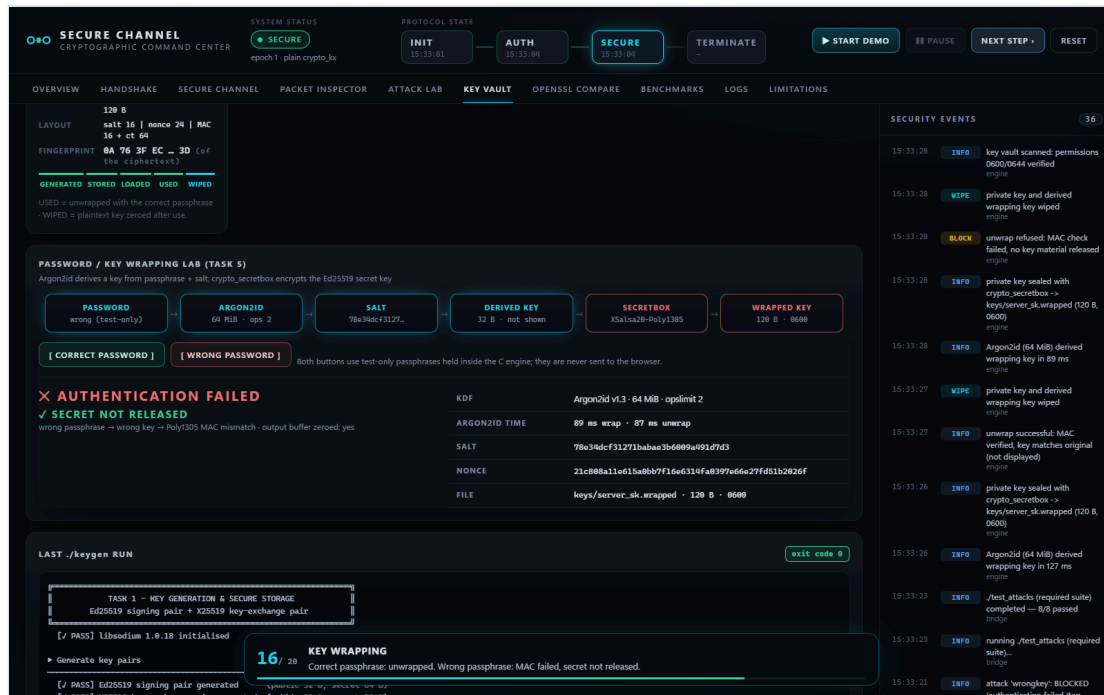
Test	Expected
Correct passphrase	Unwrapped, matches the original key
Wrong passphrase	Rejected
Tampered wrapped file (extra)	Rejected
Two wraps of the same key differ (extra)	Yes — fresh salt and nonce

11.4 Why Argon2id and what it costs

One derivation took about **130 ms** on the test machine. That is nothing for one login, but it multiplies the cost of every guess, and the 64 MiB memory requirement stops GPUs from running thousands of guesses cheaply in parallel.

ABOUT THE TEST PASSPHRASES

The automated tests and dashboard buttons use two built-in passphrases marked **TEST-ONLY** in `wrap.h` . They are never printed or sent to the UI (the integration test checks this). For real use, `./keywrap --interactive` reads your own passphrase with terminal echo off.



Key Wrapping view: wrong passphrase rejected, secret not released, output buffer zeroed.

12 Task 6 — OpenSSL comparison

Script: `scripts/openssl_compare.sh` (run by `make openssl`). It runs the brief's OpenSSL commands and then attacks the result.

12.1 The eight sections of the script

- Tool version** — `openssl version`.
- Generate keys** — `openssl genpkey -algorithm ED25519`, `pkey -pubout`, `genpkey -algorithm X25519`.
- PEM format** — shows the Base64 text between `-----BEGIN PUBLIC KEY-----` lines. The DER public key is 44 bytes (vs 32 raw bytes in libsodium); the Ed25519 private key is 48 bytes of PKCS#8 (vs 64 raw bytes in libsodium).
- File permissions** — OpenSSL 3.5.5 created the private PEM as 0600, but it is **not encrypted** unless you ask for a passphrase cipher.
- aes-256-cbc encrypt/decrypt** — the brief's `openssl enc -aes-256-cbc -salt -pbkdf2` round-trip.
- Tamper attack on CBC** — the important part (below).
- The same attack on this project's AEAD** — rejected with rc -2.
- Comparison table.**

12.2 The CBC bit-flip, explained slowly

The script encrypts `FROM=Pratham;TO=Bob;AMOUNT=00100`. AES works on 16-byte blocks:

```
block 0: FROM=Pratham;TO=
block 1: Bob;AMOUNT=00100
```

In CBC decryption, *plaintext block i* = *Decrypt(ciphertext block i) XOR ciphertext block $i-1$* . So if you flip a bit in ciphertext block 0, the **same bit flips in plaintext block 1**. The script XORs byte 11 of ciphertext block 0 with `0x09`. In block 1, byte 11 is the first `0` of `00100`: `'0'` (`0x30`) XOR `0x09` = `0x39` = `'9'`.

```
decrypted block 0: 16 random-looking bytes (collateral damage)
decrypted block 1: Bob;AMOUNT=90100 (100 became 90100)
openssl enc -d exit code: 0 (no error!)
```

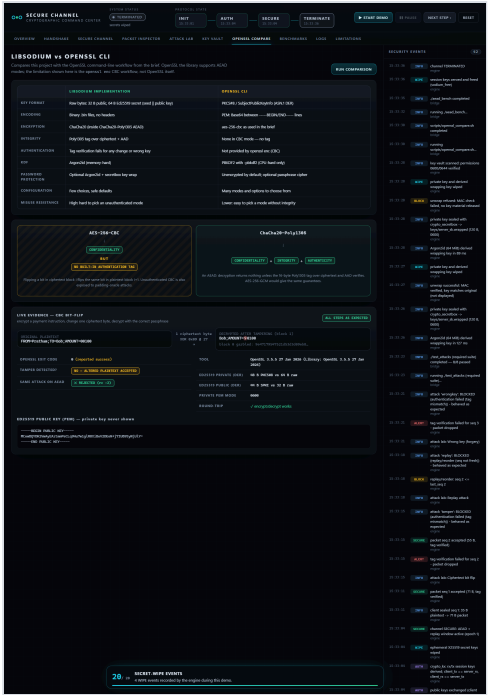
OpenSSL reports success because CBC has **no integrity check**. The same kind of change against this project's AEAD is rejected with **-2**.

12.3 Comparison table

Property	libsodium (this project)	OpenSSL CLI (as in the brief)
Key format	Raw bytes (32 B public, 64 B Ed25519 secret)	PKCS#8 / SubjectPublicKeyInfo (ASN.1 DER)
Encoding	Binary <code>.bin</code> , no headers	PEM: Base64 + BEGIN/END lines
Private-key protection	0600 via <code>open()</code> + <code>fchmod()</code> ; optional Argon2id wrap	0600 observed in OpenSSL 3; unencrypted by default
Message encryption	ChaCha20-Poly1305 AEAD	<code>aes-256-cbc</code> : confidentiality only
Tamper detection	Yes — 16-byte tag	No — bit-flip decrypted with exit 0
Password KDF	Argon2id (memory-hard)	PBKDF2 (CPU-hard only)
Misuse resistance	High — few choices, safe defaults	Lower — easy to pick an unauthenticated mode

BE FAIR TO OPENSSL

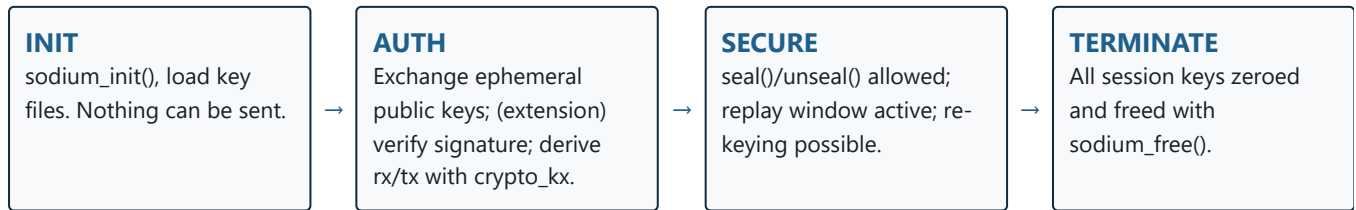
OpenSSL as a library is not insecure: its C API supports AEAD modes like AES-256-GCM and ChaCha20-Poly1305. The weakness belongs to the `openssl enc CBC` workflow in the brief.



OpenSSL Compare view with the live CBC bit-flip evidence.

13 The session state machine

A channel moves through four states. The rules are enforced in one place, `src/session.c`, which the engine, the terminal demo and the tests all share.



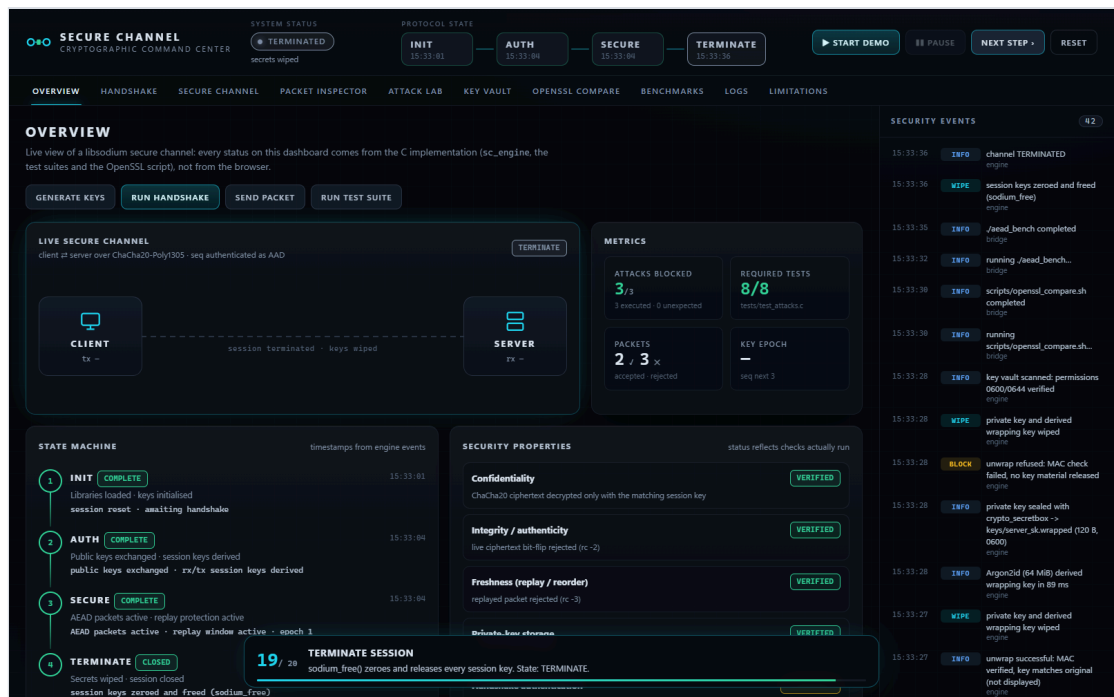
13.1 The `session` struct

- `state` — the current state.
- `c_tx`, `c_rx`, `s_tx`, `s_rx` — four 32-byte session keys in `sodium_malloc()` memory.
- `next_seq` — the client's send counter (starts at 1).
- `server_rs` — the server's replay window.
- `epoch`, `rekey_every`, `in_epoch` — re-keying (epoch 1 after the handshake).
- `authenticated` — 1 if the signed handshake was used.
- `client_pk`, `server_pk` — the ephemeral public keys (public data).

13.2 Functions

Function	Behaviour
<code>session_handshake(s, id, hr)</code>	<code>id == NULL</code> : plain kx (as in the brief). <code>id != NULL</code> : the server signs the transcript and the client verifies. Checks the keys are complementary and directional. Any failure frees the keys and returns to INIT.
<code>session_seal()</code>	Refused with -10 (<code>SESSION_ERR_STATE</code>) unless SECURE; messages over 256 B refused with -13 .
<code>session_deliver()</code>	Server side: <code>unseal()</code> with <code>s_rx</code> and the replay window; on success may trigger an automatic re-key.
<code>session_rekey()</code>	Advances all four keys to the next epoch.
<code>session_fingerprints()</code>	8-byte BLAKE2b fingerprints for display only.
<code>session_close()</code>	Frees all keys, resets the replay window, state = TERMINATE.

Extra session error codes: **-10** wrong state, **-11** crypto/allocation failure, **-12** signature did not verify, **-13** message empty or too long.



After TERMINATE: all four states have been passed and no key material remains.

14 Extension: signed handshake vs man-in-the-middle

14.1 The attack on plain `crypto_kx`

Plain key exchange proves that you share a key with *someone*, not *who*. Mallory sits in the middle and replaces each side's public key with her own. She now runs two separate exchanges: one with the client, one with the server.

- The client sends `"PAY BOB 100"` — Mallory decrypts and **reads** it.
- Mallory sends `"PAY MAL 900"` to the server — the server **accepts** it.

`make mitm` (and the dashboard Handshake view) show this happening with real keys.

14.2 The fix

The server signs a **transcript** with its long-term Ed25519 key from Task 1:

```
transcript = "SC-HANDSHAKE-v1" || server_ephemeral_pk || client_ephemeral_pk
signature  = Ed25519_sign(server_sk.bin, transcript)
```

The client verifies with `crypto_sign_verify_detached` against the `server_pk.bin` it already trusts (it is "pinned"). If verification fails, the handshake aborts.

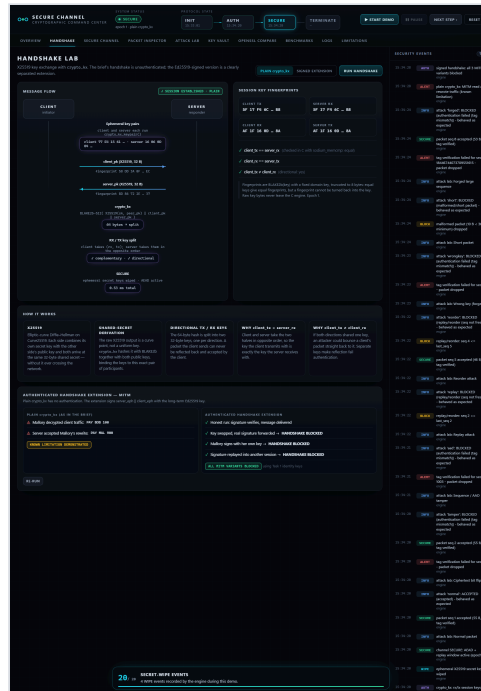
- The **label** separates this signature from any other use of the key.
- The **server key** in the transcript means Mallory cannot swap in her own ephemeral key.
- The **client key** in the transcript stops a signature from one session being replayed into another.

14.3 Three MITM variants, all blocked

Variant	What Mallory tries	Result
---------	--------------------	--------

Key swap	Substitutes her ephemeral key, forwards the server's real signature	Blocked — signature does not cover her key
Own signature	Signs with her own Ed25519 key	Blocked — not the pinned key
Replayed signature	Re-uses a valid signature in a different session	Blocked — different client key in transcript

An honest signed session still verifies, agrees on keys and delivers the message. The remaining assumption: the client must get `server_pk.bin` authentically.



Handshake view: MITM succeeds against plain kx; the signed extension blocks all three variants.

15 Extensions: re-keying, TCP and benchmark

15.1 Re-keying

Each session key is replaced in place:

```
new_key = crypto_kdf_derive_from_key(32 bytes, subkey_id = epoch, context = "SCREKEY1", old_key)
old_key is overwritten; the temporary buffer is wiped
```

- Both sides run the same function on the same old key with the same epoch, so they stay in sync **without sending anything**.
- The KDF is one-way: a key leaked in epoch n does not reveal earlier epochs.
- Triggered on demand (dashboard button) or automatically every N accepted messages (`config rekey_every N`, or `sc_server --rekey N`).
- Tested: a packet sealed under the old key fails after re-keying.

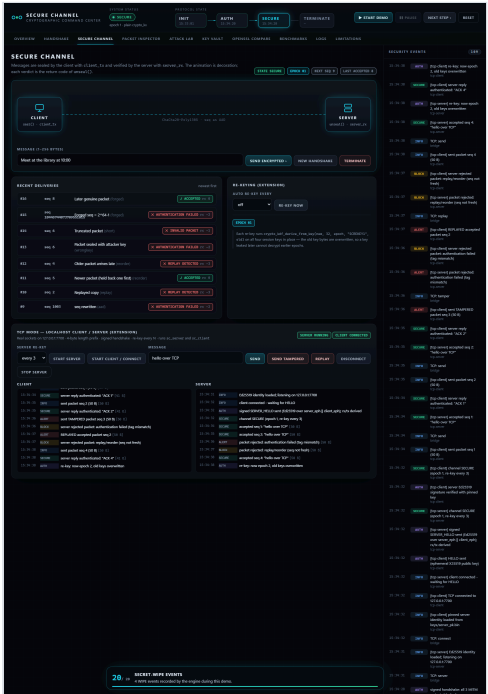
15.2 TCP client/server (`sc_server` , `sc_client`)

The secure channel over real sockets on **127.0.0.1:7700** (the server binds to localhost only).

Frame format: [length 4 B big-endian][type 1 B][body] . The length is checked against a maximum *before* anything is read.

Type	Direction	Body
1 HELLO	client → server	client ephemeral public key (32 B)
2 SERVER_HELLO	server → client	server ephemeral pk (32) signature (64) rekey_every (4)
3 DATA	both ways	a sealed packet (same format as Task 3); the server replies with a sealed ACK
4 BYE	client → server	close
5 ALERT	server → client	1 byte: negated <code>unseal()</code> rc (demo convenience)

Client commands (typed into `sc_client`): `send <text>`, `tamper <text>` (flips one ciphertext bit), `replay` (re-sends the last accepted packet), `quit` . On BYE or disconnect both sides free their keys with `sodium_free` (WIP events). A client pinned to a different identity aborts the handshake. When stdout is not a terminal, both programs print JSON lines that the bridge streams to the dashboard.



TCP mode in the Secure Channel view: signed handshake, re-keying, tamper and replay rejected over real sockets.

15.3 AEAD benchmark (`aead_bench`)

Times ChaCha20-Poly1305-IETF, AES-256-GCM (only if the CPU supports it) and the project's full `seal()` for message sizes 64, 256, 1024, 4096 and 16384 bytes, using `CLOCK_MONOTONIC` . `--json` is used by the dashboard. Example figures from the report (1 KiB, CPU with AES-NI; vary per run):

Algorithm	Time per op	Throughput
ChaCha20-Poly1305-IETF	≈ 1.3 μs	≈ 720–760 MiB/s
AES-256-GCM (AES-NI)	≈ 0.8 μs	≈ 1160–1200 MiB/s
<code>seal()</code> incl. random nonce + header	≈ 1.35 μs	≈ 710–730 MiB/s

Interpretation: AES-GCM wins on a CPU with hardware AES instructions. ChaCha20 is the safer portable default: fast and constant-time in plain software on any device. Building the packet and generating the nonce add only a few percent.

16 sc_engine: the machine interface

`sc_engine` (`src/engine.c`) is the bridge between the dashboard and the cryptography. It is one long-running process that holds a live `session` in memory.

16.1 Protocol

One command per line on standard input → one JSON object per line on standard output. On start it prints a `ready` line with the libsodium version.

Command	Effect
<code>hello , status</code>	Version and current session state, fingerprints, counters.
<code>vault</code>	Scan <code>keys/</code> : sizes, modes, and whether each private key matches its public key.
<code>handshake plain signed</code>	Run the handshake; returns public keys, fingerprints, complementary/directional checks, signature result.
<code>send <text> (1–256 B)</code>	Client seals, server delivers; returns the packet's public bytes, rc, verdict, decrypted text.
<code>attack <8 names></code>	Runs one attack (Chapter 10).
<code>rekey , config rekey_every <0..1000></code>	Manual re-key / automatic re-key interval.
<code>wrap correct wrong</code>	Task 5 with the TEST-ONLY passphrases; returns whether the secret was released and the buffer zeroed.
<code>mitm</code>	Runs the MITM scenario report.
<code>terminate , reset , quit</code>	Close session / fresh start / exit.

16.2 What the engine never outputs

No private key, no session key, no passphrase. Only public keys, nonces, ciphertext, return codes, file metadata, timestamps, and fingerprints made with `BLAKE2b(key, domain = "SC-FINGERPRINT-1")` truncated to 8 bytes. Equal keys give equal fingerprints, so the dashboard can show `client_tx = server_rx`, but a fingerprint cannot be turned back into a key. Each response also carries an event log (INFO, BLOCK, ERROR, WIPE ...) built only from public data.

16.3 json.c

A small writer into a fixed-size buffer: objects, arrays, strings (escaped), integers, booleans, and hex for byte arrays. No dynamic allocation and no external library.

17 The Node bridge and its security

`bridge/server.js` is a zero-dependency Node.js HTTP server. On start-up it runs `make -s all` (stops if the build fails), detects the gcc, libsodium and OpenSSL versions, starts `sc_engine` , listens on **127.0.0.1:8787** and opens the browser.

17.1 API

Endpoint	Purpose
----------	---------

GET / and static files	Serves dashboard/ ; path traversal outside it is refused (403).
GET /api/health	Engine ready?, libsodium version, platform, TCP status, detected toolchain.
GET /api/stream	Server-Sent Events: engine status, run notifications, TCP log lines.
POST /api/engine	{cmd, arg} → validated → one line to sc_engine → JSON back. Commands are queued so answers stay in order; 30 s timeout. The engine is auto-restarted if it exits.
POST /api/run/<what>	One-shot runs: build , keygen (+ ls -l keys/), tests , tests-extended (TAP parsed), make-test , openssl (--json), bench . Each response lists the exact commands executed.
POST /api/tcp/<action>	start-server (with re-key N), connect , send (mode send/tamper/replay), disconnect , stop .

17.2 Security controls — why another website cannot use your dashboard

Control	What it stops
Bind to 127.0.0.1 only	Other machines on the network cannot connect.
Host header must be 127.0.0.1:8787 or localhost:8787 (else 421)	DNS-rebinding attacks, where an evil domain is re-pointed to 127.0.0.1.
Every POST needs X-SC-Client: dashboard and JSON content type (else 403)	Cross-site request forgery: browsers cannot send a custom header cross-origin without a CORS preflight, which the bridge never approves.
Command whitelist + validation	Only fixed command names; attack/handshake/wrap arguments from fixed lists; re-key 0–1000; messages 1–256 bytes with control characters removed; body limited to 4 KiB. Something like attack rm -rf is rejected (400).
Strict Content-Security-Policy	script-src 'self' , frame-ancestors 'none' etc.: no injected or external scripts, no framing.
Other headers	X-Content-Type-Options: nosniff , X-Frame-Options: DENY , Referrer-Policy: no-referrer , Cache-Control: no-store .
No secrets as arguments	Secrets never appear in command lines, URLs, the DOM or browser storage.

On Ctrl+C the bridge sends quit to the engine, stops TCP processes and exits.

18 The dashboard and Demo Mode

A static, dark "liquid-glass" console. Two strict rules govern it:

RULE 1 — NO ANIMATION

No keyframes, transitions, spinners, pulsing, smooth scrolling or moving packets. Hover and click change appearance instantly. app.css has a global animation/transition: none !important rule, and the integration test checks the stylesheet.

RULE 2 — NO FAKE RESULTS

Every PASS, VERIFIED or number comes from a real C/bridge result in the current session. Until something runs, the UI shows **NOT RUN**. Nothing is hard-coded from the README.

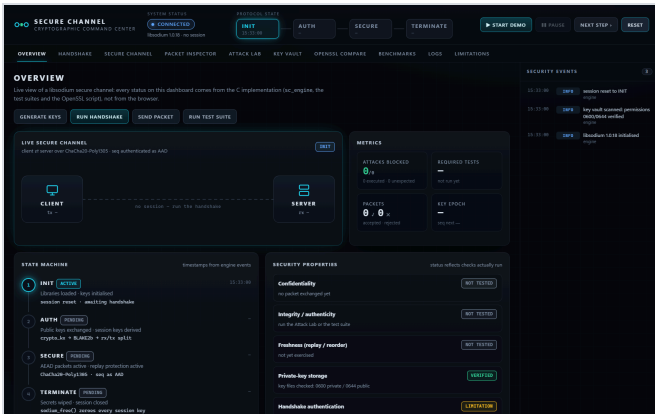
18.1 The 12 views

View	What it shows / does
Overview	Engine status, toolchain versions, state machine, security properties, summary of what has run.
Handshake	Plain or signed handshake, ephemeral public keys, session-key fingerprints, MITM comparison.
Secure Channel	Send messages, see the packet and verdict; re-key controls; TCP mode panel.
Packet Inspector	Byte-level breakdown of a packet: seq (AAD), nonce, ciphertext, tag.
Attack Lab	Launch each of the 8 attacks or RUN ALL 8; steps, rc, verdict, reason.
Test Center	Run the required (8) and extended (35) suites and <code>make test</code> ; per-test results.
Key Vault	GENERATE KEYS (runs <code>./keygen</code>), file modes, sizes, key-pair consistency.
Key Wrapping	Correct vs wrong passphrase (TEST-ONLY), secret released or not, buffer zeroed.
OpenSSL Compare	Runs the script in JSON mode; PEM sizes, modes, CBC bit-flip evidence, AEAD rejection.
Benchmarks	Runs <code>aead_bench --json</code> ; static SVG charts of measured throughput.
Logs	All events with filters (e.g. WIPE events after TERMINATE).
Limitations	The honest list of what the design does not do (Chapter 22).

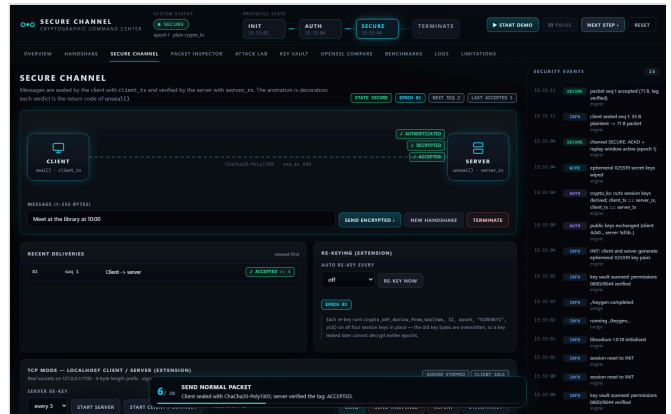
18.2 Demo Mode — 20 steps

START DEMO runs a guided presentation; **PAUSE**, **NEXT STEP** and **RESET** control it. Each step calls the real engine or binaries, and the narration that reports a result is computed from the actual response.

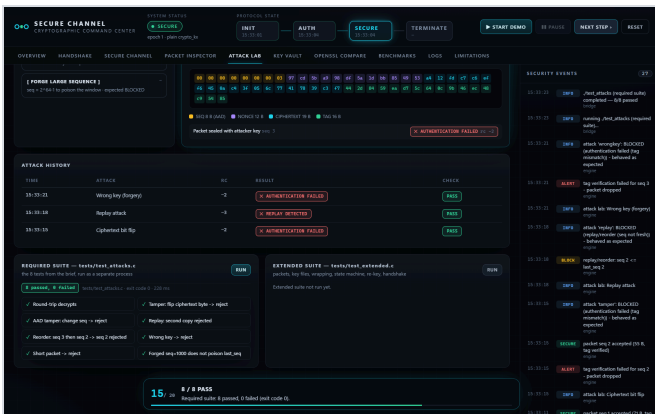
#	Step	#	Step
1	System overview (engine reset)	11	Replay rejected
2	Key generation (<code>./keygen</code> , vault)	12	Wrong-key forgery
3	Key exchange	13	Forgery rejected
4	Derive TX / RX keys	14	Full test suite
5	Enter SECURE state	15	Test results (title shows the real pass count, e.g. 8 / 8 PASS)
6	Secure message	16	Key wrapping
7	Packet inspection	17	OpenSSL comparison
8	Launch tamper attack	18	Benchmark
9	Tamper rejected	19	Terminate session
10	Launch replay attack	20	Secret-wipe events



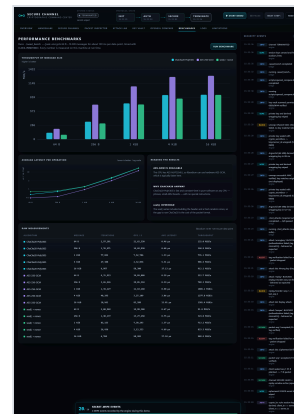
Overview



Secure Channel



Test Center



Benchmarks

18.3 The terminal demo (secure_demo)

For presenting without a browser: `./secure_demo` (menu), `--present` (guided tour, Enter between steps), `--auto` (no pauses), `--audit` (16-row security audit that exits 0 only if every check passes — it runs the attack tests, checks key modes, checks no keys are tracked by git, and runs the OpenSSL script).

19 Testing: four suites

19.1 Required — tests/test_attacks.c (8 tests)

The brief's Task 4 tests (Chapter 10). Run by `make test`; output must read **8 passed, 0 failed**. `--tap` gives `ok 1 ...` lines that the bridge parses.

19.2 Extended — tests/test_extended.c (35 tests)

Area	Examples of what is checked
Packets	Big-endian encode/decode; packet length = 20 + pt + 16; seq readable in clear; <code>seal()</code> refuses too-small buffer; <code>unseal()</code> returns -4 for small buffer, -1 for oversized packet; failed tag gives rc -2, zero length, zeroed output.
Key files	<code>write_file_0600</code> creates 0600; an existing 0644 file is tightened to 0600; wrong-size file rejected; exact bytes loaded; planted symlink refused (<code>O_NOFOLLOW</code>).

Wrapping	Correct passphrase unwraps; wrong rejected and nothing released; tampered box rejected; file is 120 B and 0600.
State machine	Starts in INIT; seal refused before handshake; handshake reaches SECURE; >256 B refused; round trip; fingerprints match; TERMINATE frees keys; deliver refused after TERMINATE.
Re-key	Epoch advances; keys change but sides still match; old-key packet rejected; new-epoch round trip; automatic re-key after N messages.
Handshake	Signature verifies under pinned key; substituted key fails; different pinned key fails; signed session reaches SECURE; plain kx MITM succeeds (documented); signed blocks all MITM variants.

19.3 Integration — `bridge/test_integration.js` (46 checks)

Starts the real bridge and drives it over HTTP like the browser does:

- **Start-up and page:** bridge builds C and starts the engine; page served; stylesheet has no keyframes or transitions; CSP present.
- **Security controls:** POST without header → 403; foreign Host → 421; path traversal blocked; non-whitelisted command, invalid attack name and oversized message → 400.
- **Engine:** reset → INIT; attack refused outside SECURE; handshake → SECURE with complementary and directional keys; fingerprints match; message delivered; packet layout 8/12/16 B; **all 8 attacks** give the expected rc; manual and automatic re-key.
- **Runs:** required suite 8/8; extended suite all pass; keygen + `-rw-----`; vault modes and pair checks; wrap correct/wrong; signed handshake; MITM report; OpenSSL CBC tamper undetected + AEAD rejects; benchmark returns measured data.
- **TCP:** signature verified with pinned key; messages accepted; re-key on both sides; tampered and replayed packets rejected; keys wiped on disconnect.
- **Final:** terminate leaves no key material; **no private-key bytes and no passphrase in any response or event.**

19.4 OpenSSL script

`make check` runs it with `--quiet` and prints "OpenSSL comparison: OK" only if every step behaved as expected (including the CBC tamper going undetected).

20 How to run everything

20.1 One-time setup (Ubuntu / WSL)

```
sudo apt update
sudo apt install -y build-essential libsodium-dev libssl-dev pkg-config openssl git
# On a Windows drive, also: [automount] options = "metadata" in /etc/wsl.conf, then wsl --shutdown
```

20.2 Terminal (inside WSL, in the project folder)

```
make env          # print tool versions
make              # build all 11 programs
make test         # brief checkpoints: keygen, kx_demo, test_attacks (8/8), keywrap
make check        # + 35 extended tests + OpenSSL comparison
make openssl      # Task 6 with full output
make mitm         # MITM on plain kx, then the signed fix
make benchmark    # AEAD speeds
make present      # guided terminal tour
make audit        # security audit
make tcp-server   # terminal 1
make tcp-client   # terminal 2: send hello | tamper hi | replay | quit
./keywrap --interactive # wrap the key with your own passphrase
```

From Windows PowerShell you can run the same thing through WSL:

```
wsl -d Ubuntu -- bash -lc 'cd "/mnt/d/project related 24 oct/secure-workshop" && make check'
```

20.3 Dashboard (Windows, Node ≥ 18)

```
cd "D:\project related 24 oct"      # or the secure-workshop folder
npm run dev                          # builds, starts the bridge, opens http://localhost:8787
npm run test:ui                      # 46 integration checks
npm run check                        # make check in WSL + integration tests
```

- **START DEMO** runs the 20-step presentation. Each tab also works on its own; the Attack Lab's **RUN ALL 8** fires every attack live.
- Stop with Ctrl+C. On Linux/macOS run `node bridge/server.js`.

NOTE

`make audit`, Demo Mode and the vault's GENERATE KEYS button re-run `./keygen`, which replaces the keys in `keys/`; the key is then re-wrapped with the TEST-ONLY passphrase.

20.4 Suggested live-demo script (about 4 minutes)

1. START DEMO: keys generated, 0600 verified in the Key Vault.
2. Handshake: point at the matching fingerprints.
3. Send a packet: AUTHENTICATED, DECRYPTED, ACCEPTED.
4. Tamper, replay and wrong key: each BLOCKED with its return code.
5. Test suite 8/8, key wrapping, OpenSSL CBC evidence.
6. TERMINATE, then Logs filtered to WIPE events.

Backup if WSL or the browser fails: `make present` in a terminal, or the screenshots in `docs/screenshots/`.

21 Design questions and answers

Short answers you can say out loud, with the file to point at.

Q1. Why are private keys stored with 0600 permissions, created atomically?

0600 means only the owner can read or write. Creating the file with that mode in `open()` means it never exists with looser permissions. With `fopen()` + `chmod()`, another user could open it during the gap and keep reading

through the file descriptor. (`src/keyio.c`)

Q2. Why does `crypto_kx` hash the shared secret with both public keys?

The raw X25519 output is a curve point, not a uniform key, so it is hashed (BLAKE2b-512). Including both public keys binds the keys to *this exact pair* — a substituted key gives different keys — and defends against key-substitution tricks. The 64 bytes are split into two direction keys, taken in opposite order by each side, which is why `client_tx == server_rx`.

Q3. Why is the sequence number sent in clear but included as AAD?

The receiver needs it before decrypting to reject replays cheaply. As AAD it is covered by the tag, so changing it (test 3) makes authentication fail; old packets cannot be "renumbered".

Q4. Why must `last_seq` be updated only after the tag verifies?

Before verification, `seq` is attacker-controlled. Updating first would let one forged packet with `seq = 264 - 1` block all later genuine traffic (denial of service). Test 8 and the live "forged" attack prove the state is unchanged. (`src/aead.c`)

Q5. Which requirement does `aes-256-cbc` fail?

Integrity/authenticity. Flipping a bit in ciphertext block i flips the same bit in plaintext block $i+1$; `00100` became `90100` with exit code 0. CBC without a MAC is also open to padding-oracle attacks. Fix: an AEAD, or at least encrypt-then-MAC.

Q6. Why `sodium_memzero()` instead of `memset()` ?

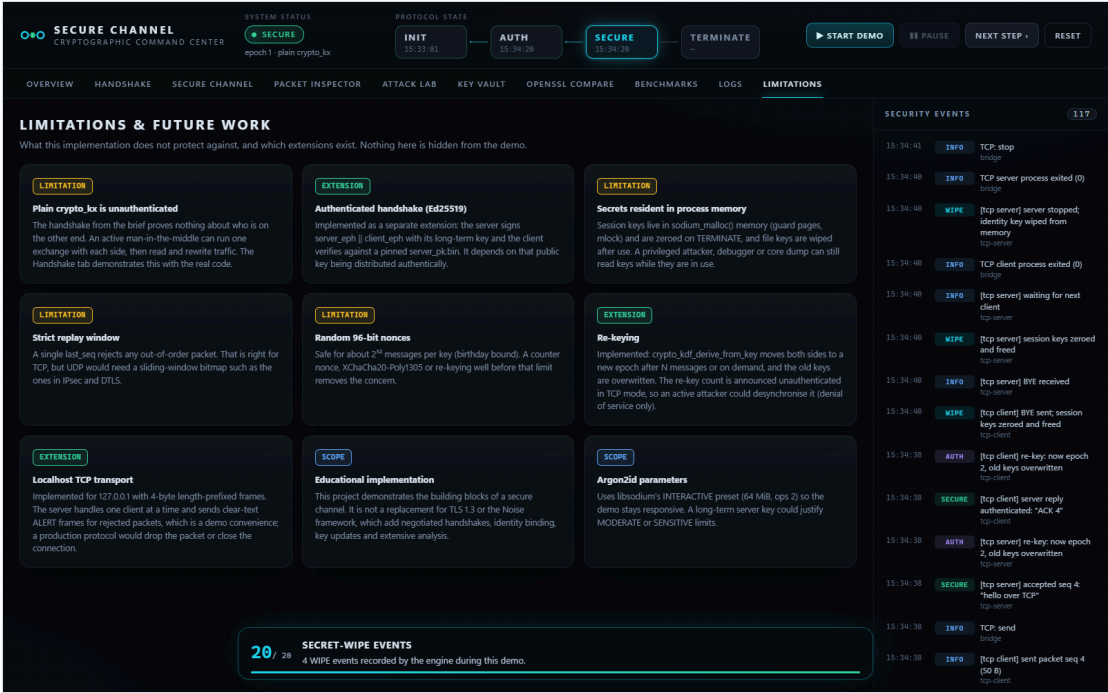
The compiler may delete a `memset` on a buffer never read again (dead-store elimination). `sodium_memzero` cannot be removed.

Other likely questions

Question	Answer
Why must a nonce not repeat?	Same nonce + key = same keystream; XOR of ciphertexts reveals XOR of plaintexts, and Poly1305 becomes forgeable.
Why does wrong-key decryption fail?	The receiver recomputes the tag with its own key; a different key gives a different tag, so nothing is returned.
How likely is a forged tag?	Negligible — for messages this size, far below 1 in 2^{100} per attempt.
Why ChaCha20 and not AES?	Fast and constant-time on any CPU without AES hardware. AES-GCM is faster here only because of AES-NI.
What happens near 2^{64} messages?	The counter would wrap; with random 96-bit nonces you should re-key well before 2^{32} messages anyway.
How do you show keys without leaking them?	8-byte BLAKE2b fingerprints with a fixed domain key: equal keys → equal fingerprints, but irreversible.
Can another website drive the dashboard?	No: localhost only, custom header required (CSRF), Host check (DNS rebinding), fixed command list.
Is the dashboard just a simulation?	No. Every result is a return code from the C code via the bridge; nothing is precomputed, and nothing moves on screen.
Is this production-ready?	No. It is educational; production systems use TLS 1.3 or the Noise framework.

22 Limitations

- **Plain `crypto_kx` is unauthenticated.** An active MITM can read and rewrite traffic (demonstrated). The signed extension fixes this only if `server_pk.bin` was distributed authentically.
- **Secrets are in process memory while in use.** Session keys use `sodium_malloc` (mlock, guard pages) and are zeroed on terminate, but a privileged attacker or a core dump could still read them.
- **Strict replay window.** A single `last_seq` drops every out-of-order packet — fine for TCP, too strict for UDP, which would need a sliding bitmap (as in IPsec/DTLS).
- **Random 96-bit nonces** are safe up to about 2^{32} messages per key (birthday bound). Counter nonces, XChaCha20-Poly1305 or re-keying remove the limit.
- **TCP mode:** `rekey_every` is announced unauthenticated (an attacker could only cause denial of service); rejected packets get a clear-text ALERT frame as a demo convenience; one client at a time.
- **Argon2id parameters** use the INTERACTIVE preset; a long-term server key could justify MODERATE or SENSITIVE.
- **Educational implementation**, not a TLS 1.3 or Noise replacement.



The Limitations view in the dashboard states the same points.

23 Quick-reference cheat sheet

Sizes

Session / AEAD key	32 B
Sequence number	8 B (big-endian)
Nonce (ChaCha20-Poly1305-IETF)	12 B
Tag	16 B
Header / minimum packet	20 B / 36 B

Return codes

0	accepted
-1	short / malformed
-2	authentication failed
-3	replay / reorder
-4	output buffer too small

Max plaintext (seal / session)	64 KiB / 256 B	-10	wrong session state
Ed25519 pk / sk / signature	32 / 64 / 64 B	-11	crypto / allocation failure
X25519 pk / sk	32 / 32 B	-12	handshake signature failed
Wrapped key file	120 B (16+24+16+64)	-13	message empty or too long
Fingerprint	8 B BLAKE2b		
Argon2id memory	64 MiB		

Algorithms and libsodium calls

Job	Algorithm	Call
Identity	Ed25519	<code>crypto_sign_keypair</code> , <code>crypto_sign_detached</code> , <code>crypto_sign_verify_detached</code>
Key exchange	X25519 + BLAKE2b	<code>crypto_kx_keypair</code> , <code>crypto_kx_client/server_session_keys</code>
Packets	ChaCha20-Poly1305-IETF	<code>crypto_aead_chacha20poly1305_ietf_encrypt/decrypt</code>
Password KDF	Argon2id	<code>crypto_pwhash</code>
Key wrap	XSalsa20-Poly1305	<code>crypto_secretbox_easy</code> / <code>_open_easy</code>
Re-key	BLAKE2b-based KDF	<code>crypto_kdf_derive_from_key(..., "SCREKEY1")</code>
Fingerprint	BLAKE2b	<code>crypto_generichash</code> (key "SC-FINGERPRINT-1")
Randomness	OS CSPRNG	<code>randombytes_buf</code>
Hygiene	—	<code>sodium_memzero</code> , <code>sodium_memcmp</code> , <code>sodium_malloc/free</code>

The 8 attacks in one line each

normal → 0 · tamper → -2 · aad → -2 · replay → -3 · reorder → -3 · wrongkey → -2 · short → -1 · forged → -2 (and state unchanged, later genuine packet accepted)

References

1. libsodium documentation — <https://doc.libsodium.org/>
2. RFC 8439 — ChaCha20 and Poly1305 for IETF Protocols
3. RFC 9106 — Argon2 Memory-Hard Function
4. RFC 7748 — Elliptic Curves for Security (X25519)
5. RFC 8032 — Edwards-Curve Digital Signature Algorithm (Ed25519)
6. OpenSSL 3 manual pages: `openssl-genpkey(1)`, `openssl-enc(1)`
7. S. Vaudenay, *Security Flaws Induced by CBC Padding*, EUROCRYPT 2002